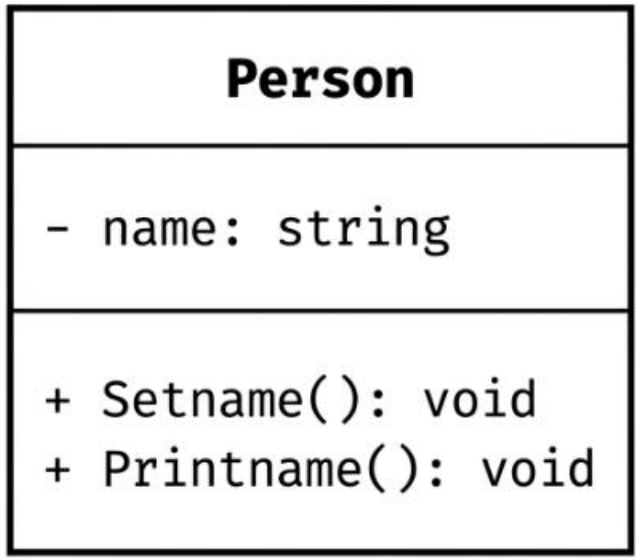
Lab5



Defining the Person Class Blueprint

- **Encapsulated Data**

  The class contains a single private data item, `name`, which holds a string representing a person's name.

- **Public Interfaces**

  Two member functions govern data access:

  - `setName()`: Allows the internal name string to be modified.
  - `printName()`: Displays the stored name to the user.

Orchestrating the Client Code Execution

Phase 1: Initialization
Define **`persPtr`**, an array explicitly sized for 100 pointers to the **`person`** type (Note: This is an array of pointers, not an array of objects).

↓

Phase 2: Input Iteration
Execute a **do-loop** that prompts the user to enter a name.

↓

Phase 3: Dynamic Instantiation
With the provided name, create a **new `person`** object **dynamically** using the **`new` keyword**. Store the pointer to this **newly** created object **sequentially** in the **`persPtr`** array.

↓

Phase 4: Output & Verification
Iterate through the **`persPtr`** array to access the objects via **their pointers**. Print out the name data for each **`person`** object to demonstrate successful data retrieval.

What is wrong with the following program?.

```
int main( ){
      int x =20;
      int* p;
      p = new int;
      *p = 30;
      cout << *p << " " << x << endl;
      delete p;
      p = &x;
      delete p;
      cout << *p << " " << x << endl;
            return 0;
}
```

What is wrong with the following program?.

```
int main( ){
            const int SIZE = 10000000;
            int i;
            int A[SIZE];

            for(i=0; i<SIZE; i++)
                  A[i] = i;
            int *B;
            B = new int[SIZE];

            for(i=0; i<SIZE; i++)
                  B[i] = A[i];

            for(i=0; i<SIZE; i++)
                  cout << A[i] << " " << B[i] << endl;
                 return 0;
}
```

What is wrong with the following program?

```
void F1(int* temp){ *temp = 99; }
void main( ){
      int *p1, *p2;
      p1 = new int;
      *p1 = 50;
      p2 = p1;
      F1(p2);
      cout << *p1 << " " << *p2 << endl;
      p1 = new int;
      *p1 = 88;
      cout << *p1 << " " << *p2 << endl;
      delete p1;
      delete p2;
}
```

What is wrong with the following program?.

```
int main( ){
      int i, *p, *r, t[4]={0,1,2,3};

            p = new int[4];
            for(i=0; i<4; i++)
              p[i] = 6*(i+1);
            r = t;
            for(i=3; i>=0; i--)
              r[i] -= 1;
            r[2] = 8;
            for(i=0; i<4; i++)
              t[i] = p[i] + r[i];
            for(i=0; i<4; i++)
              cout << t[i] << " ";
            delete [] p;
           return 0;
      }
```

What is wrong with the following program?.

```
int main( ){
      int *p, *r;
      p = new int;
      r = new int;
      p = 1;
      r = p;
     return 0;
}
```

- Write simple code for the following:

Define a dynamic array called **Darr** of 500 integers initialized to zero.

You have a **Distance** class that has **feet** as int, and **inches** as float, Based on this:

2. Define an overloaded constructor that sets **ft** parameter to feet and **inc** param to inches. (Initialization Must be in the same line using **initializer list** not in the body of constructor).

3. Define an overloaded constructor (with **default values** 1 and 1) that set **feet** parameter to feet attribute and **inches** param to inches attribute. (initialization Must be in the body of constructor).

4. Define an array of **3 Distance class objects**, initializing each one using the overloaded constructor with the values (2,3), (5,3), and (7,3) in the **initializer list**.